Author Proof

Anomaly Detection in Industrial IoT Applications Using Deep Learning Approach

Chetan Bulla and Mahantesh Birje

Abstract Internet of Things (IoT) becomes popular in last two decade as it provides many advantages such as flexibility, autonomous, cost effective and productivity. Many industries adopted IoT to improve the efficiency, security and predictive maintenance. To improve the quality of service (QoS), it is essential to identify various types of anomalies in productive maintenance. An anomaly is a value, a status of resources or outcome that deviates from expected or normal values and it affects QoS of production. In this paper, a multi-agent based anomaly detection scheme is introduced to improve the QoS. The fog computing infrastructure is used to reduce the latency of communication. Multiple agents are deployed in fog node to perform the various operations of detecting anomalies. The proposed anomaly detection scheme uses a multi-step prediction technique and applies an anomaly detection algorithm to detect anomalies. The Gated Recurrent Unit (GRU) model is used for Multi-step time series prediction and a bio-inspired Artificial Bee Colony algorithm is used for tuning the GRU model hyperparameters to improve the accuracy. The proposed model detects various types of anomalies in the fog computing environment. The Google Colab using TensorFlow library Keras is used for experimental evaluation. The proposed model increases accuracy over existing approaches, according to the experiment evaluation.

Keywords Multi-step prediction · Anomaly detection · Hyperparameter optimization · Prediction · GRU · Cloud computing · And fog computing

C. Bulla (✉)
Department of CSE, KLECET, Chikodi, India

M. Birje
Department of CSE, VTU, Belagavi, India

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022

S. L. Fernandes and T. K. Sharma (eds.), *Artificial Intelligence in Industrial Applications*, Learning and Analytics in Intelligent Systems 25,
https://doi.org/10.1007/978-3-030-85383-9_9

UNCORRECTED PROOF

511082_1_En_9_Chapter ☑ TYPESET ☐ DISK ☐ LE ☑ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

1 Introduction

Internet of Things (IoT) is the new emerging technology that interconnects various types of computing devices, mechanical devices, and digital machines [1]. Each of these devices contain different types of sensors that generate and transfer data over the cloud for processing without human-to-computer interaction. IoT was initially introduced for industry and business applications, but now it is used in offices, homes, and hospitals. The main reason behind this change is the usage of intelligent devices in everyday life. These intelligent devices are cheaper, easy to use, low maintenance costs, and can work in any environment. Time-sensitive IoT applications require a shorter response time and mobility support, such as smart hospitals and smart cities. Cisco has introduced the fog computing where computational resources are brought to the network edge to resolve the bandwidth and latency issue [2, 4].

The monitoring system [6] providing accurate information of fog computing resources ad associated IoT devices. It keeps track of all data flow and resources' status [8] and alerts the users for abnormal conditions. To differentiate between normal and abnormal systems, it is essential to develop an anomaly detection model. Anomaly detection is one of the major concerns in improving performance and troubleshooting activities in fog computing to improve QoS [5]. An anomaly is a value, a status of resources or outcome that deviates from expected or normal values/status. It is also helpful to identify the malfunction and anticipate failure. The major reasons for anomalies are resource competition, resource bottleneck, misconfigurations, software defects, hardware failures, and external attacks in fog computing infrastructure. Early detection of these anomalies improves the reliability and availability and reduces the shutdown time and maintenance cost.

Different types of anomaly can be found at different layers of fog computing infrastructure. Figure 1 shows the fog computing architecture with possible anomalies [7]. There are three layers: Cloud layer, Fog layer, and Terminal layer. The terminal layer contains various sensors and actuators that sense the environment and transfer the data to the fog layer. The fog layer is an intermediate layer and contains limited computational capacity such as Gateways and fog servers. The fog layer checks the processing requirements and processes the data. If the fog layer is unable to process then it is sent to the cloud layer. The cloud and fog computing infrastructure suffer from CPU hog, memory leak, and Workload contentions. The CPU hog means the processor consumes all its processing cycles by injecting infinite loops or viruses. Memory leaks suffer from memory exhausts by injecting anomalies to consume all memory capacity, and workload contention leads to saturation in resource components by emulating user requests.

Most fog-based applications use IoT devices or sensors that are highly dependent on accurate operation, which is reasonably difficult to guarantee. These sensors are inexpensive and prone to malfunction or error. In addition, sensors are installed under potentially harsh weather conditions, making their breakage much more probable [9]. The reasons for the sensor's anomalies are Spike, Drift, and constant. The Spike is a short duration in sensed values that diverges from other measurements; Drift is

UNCORRECTED PROOF

511082_1_En_9_Chapter ☑ TYPESET ☐ DISK ☐ LE ☑ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

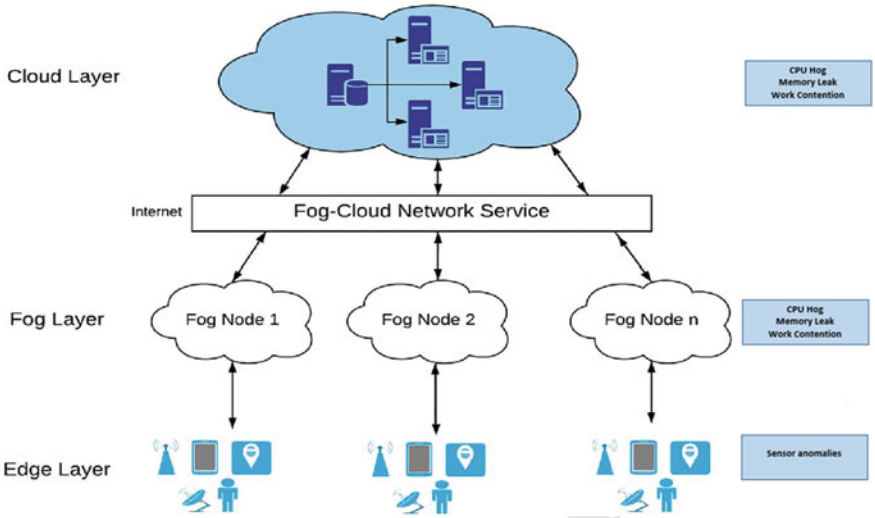

Fig. 1 Fog computing architecture with possible anomalies

off-set in the measurements of sensor values and a sensor reports as a constant value in a different scenario. The high probability of erroneous readings or data corruption occurs during data transmission between sensors and fog nodes. The large variations in sensed values and read data in fog nodes is called a sensor anomaly. The anomalies in fog computing increase shutdown time, maintenance cost, energy consumption, computational overhead and reduce performance, availability, and reliability. So it is essential to detect the anomalies to improve the performance of fog computing services. Anomalies can be classified into the following three categories:

Point anomaly are separate data points that are considered anomalous with respect to the remaining data. This type of anomaly refers to data points that are given without contextual aspects, such as time. *Collective* anomaly refers to a set of continuous data points, which are anomalous w.r.t the complete dataset. Two notes are worth mentioning: firstly, individual data points in such an anomalous collection may not be anomalies by themselves, while only their common existence shows up as an anomaly. Secondly is that this type of anomaly can appear in both context-aware data and contextless data. *Contextual anomalies* are individual data points or collection of data points are considered as anomalous with respect to particular context or applications.

In this paper, the multiple agents system (MAS) is used to perform the various operations in detecting anomalies [6]. These agents are installed in each fog node to collect the data, prepare the data, anomaly detection, and hyperparameters tuning. The agents are intelligent programs, and these programs work cooperatively and collaboratively to perform a variety of tasks. Figure 2 shows the architecture of a multi-agent system in a fog computing environment. The architecture is divided into 3 layers: cloud, fog and IoT layer. Cloud layer contains large datacenters. The fog

UNCORRECTED PROOF

511082_1_En_9_Chapter ✓ TYPESET ☐ DISK ☐ LE ✓ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

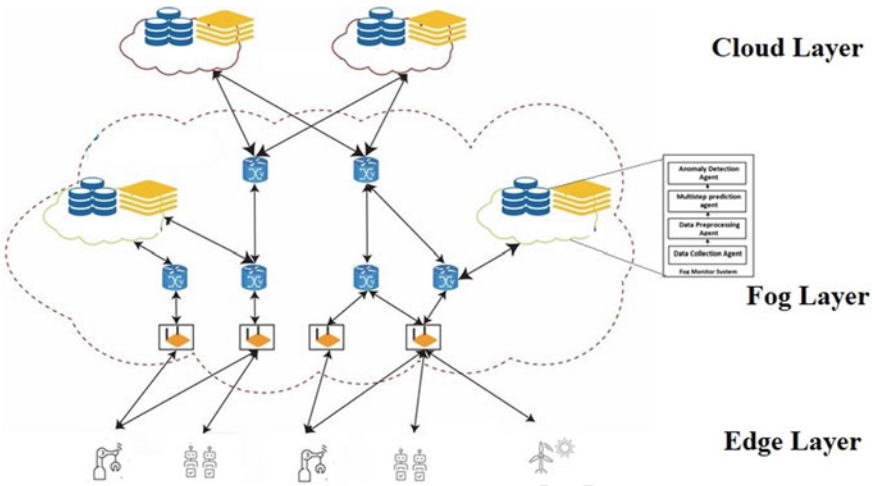

Fig. 2 Multi-agent based anomaly detection in IIoT environment

node have limited computational resources. The IoT devices contains various smart devices and its sensors. The traditional sugar cane factory slowly converting to smart industries by adapting automation in manufacturing process. The various sensors are deployed in smart machines/devices to check proper operations. The smart devices are connection to fog computing infrastructure for computational services.

Various IoT applications such as Industry IoT, Medical IoT, and smart city, etc. are used in the fog computing environment. These applications install multiple sensors and actuators. The data generated by the fog node, virtual machine, and sensors are gathered by the collection agent [3] and sends them to the data preparation agent. The data preparation agent removes outliers, noise, and normalize the data. The anomaly detection agent predicts future time series and identifies anomalies. Hyperparameter tuning for GRU model using artificial bee colony is carried out by hyperparameter optimization agent. The anomaly detection agent compares the predicted value and actual values of data. If the difference between these is more than a certain threshold, then it is identified as an anomaly. The main contributions of this proposed work are: multi-step prediction on time series data is designed using GRU Model. An Artificial Bee Colony based hyperparameter optimization algorithm is implemented to increase the accuracy of multi-step prediction. An adaptive threshold mechanism is used to identify anomaly in Industrial IoT.

The remaining sections are organized as follows: Sect. 2 discusses state of art work on anomaly detection methods. Section 3 presents anomaly detection model for fog computing environment. Section 4 discusses the performance evaluation of the proposed anomaly detection model. Finally paper is summarized and concluded in Sect. 5.

UNCORRECTED PROOF

511082_1_En_9_Chapter ✓ TYPESET ☐ DISK ☐ LE ✓ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

2 Related Work

In real-time applications, anomaly detection and prediction are of great importance in preventing anomaly or reducing damage. An adaptive anomaly detection scheme for IoT data is proposed for Hierarchical Edge Computing [4]. The author proposed three anomaly detection models to match different complexities and associate them with the HEC layer in a bottom-up approach. Hyper ellipsoidal Clustering [11] proposed fog empowered distributed computing. The fog node and cloud node divide the data based clustered and adopt an anomaly detection algorithm. The proposed solution compared two traditional anomaly detection models, (i) Centralized and (ii) Distributed algorithm. The results show a specific improvement in the accuracy of the anomaly detection model.

A lightweight anomaly detection model for Internet of Things proposed in [15]. The proposed model is host based anomaly detection model uses two popular clustering technique inverse Weight Clustering (IWC), and decision tree. The model initially divide the dataset into clusters and apply decision tree for classification. To evaluate the model, the authors used only two sensors, temperature, and humidity. IWC is a modified version of k-means, run unlimited iterations to get optimal solutions. To detect and predict anomalies at runtime to optimize system performance, Hidden Markov Model-based anomaly detection and the prediction model is developed [10]. The proposed model considered the problem of mapping observations and identified anomalous behavior edge cluster workloads. The authors extracted variations in the containers' workload over various timestamps captured by Hidden Markov Models (HMM) and it is used to calculate model parameters from container workload data.

The different machine learning models [14] for anomaly detection in IoT has been proposed. To evaluate and compare these machine learning models accuracy, and receiver operating characteristic curve are used. The ANN, Random Forest, and Decision Tree models recorded the accuracy up to 98.6%. To detect various threats and false alarms, a machine learning-based anomaly detection model is developed. A non-parametric Online Locally Weighted Projection Regression (OLWPR) is used to detect wireless sensors networks anomalies. Various parameters like recall, precision, accuracy, RMSE, and MAPE are used to evaluate the proposed model. The experiment evaluation shows that the anomaly detection model improves accuracy compared to the Dynamic Bayesian Network (DBN).

The most advanced and dynamic reinforcement learning algorithm called asynchronous advantage actor-critic (A3C) algorithm addresses the anomaly detection problem. The experiment results show improvement in recall and precision compared to other machine learning based anomaly detection models. A vector convolutional deep learning (VDCL) model is used to identify outliers in IoT. The main objective is to improve the scalability of anomaly detection in the non-scalable nature of network traffic records in IoT. The master fog node uses the VCDL model to classify the known and unknown anomalies. The experimental results showed that the proposed framework improved accuracy, precision, and recall compared with the

UNCORRECTED PROOF

511082_1_En_9_Chapter ☑ TYPESET ☐ DISK ☐ LE ☑ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

existing anomaly detection system. An anomaly detection method uses multivariate data using a convolution neural network [15] to improve accuracy. The proposed model behaves human vision and decision making to detect anomalies. CNN is trained to identify and detect anomalies in multivariate time series data. During training representation and description of anomalies were learned. Experiment results show that proposed anomaly detection is scalable, accurate, and faster.

3 Proposed Model

The various IoT devices from different locations are connected to the fog nodes. The IoT devices generate large amount of data repeatedly. These raw data is collected and apply filtering to convert the data in a appropriate format for training. The proposed anomaly detection model works in three stages (i) Data preprocessing (ii) Multistep prediction and (iii) Anomaly detection. The data preprocessing is used to prepare the data for training and testing purposes. The proposed model utilizes the fog monitoring system to collect, filter, and normalize the data coming from IoT devices. The GRU model with hyperparameter tuning is used for multistep prediction. (iii) Anomaly detection compared the actual data with predicted data and consider certain points as anomaly if the difference between actual and predicted data is more than threshold value. Figure 3 shows the proposed anomaly detection model. The following sub sections discusses the working of these stages.

1. Data Preprocessing

The quality of data is very crucial in training the deep learning. The data preprocessing is the process of removing unwanted data and making it suitable for deep learning models. The proposed system utilizes the fog monitoring system to collect, filter, and normalize the data coming from IoT devices. The monitoring system continuously keeps track of all the information of fog computing infrastructure. It monitors all resources' health, Event data, and data generated by IoT devices in fog computing.

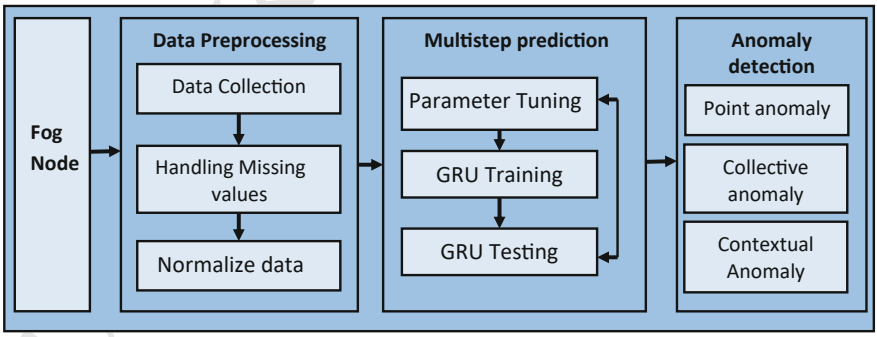

Fig. 3 Anomaly detection with hyperparameter tuning model

511082_1_En_9_Chapter ✓ TYPESET DISK LE ✓ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

The proposed model uses a fog monitoring system [3] for data preprocessing tasks such as data collection, aggregation, data filtering, handling missing values, and data normalization. The quality of data is one of the key factors for any deep learning model to improve the performance. In the proposed model, two data processing strategy (i) handling missing data and (ii) Normalize the data are adopted. To handle the missing data, the mean, median, or mode of a particular feature is calculated and added to missing values. In order to avoid collecting duplicate data frequently, the model collects the temporal data if there is a certain difference between previous data and current data.

$$CD_t = \frac{D(t) - D(t-1)}{MAX} X\,100 \tag{1}$$

The mean and median values of a particular feature is used to fill up the missing values in training data.

$$\bar{x} = \frac{\sum_{i=1}^{N} x}{N} \tag{2}$$

$$Median = \left(\frac{n+1}{2}\right)^{th} term\ (if\ \ n\ is\ even) \tag{3}$$

$$Median = \frac{\left(\frac{n}{2}\right)^{th} term + \left(\frac{n}{2}+1\right)^{th} term}{2} \text{(If n is odd)} \tag{4}$$

Further, it is essential to normalize the outliers in the data. An outlier is a data point that differs significantly from other observations. The outliers in data may reduce the performance of the deep learning model. So it is essential to normalize the data to a certain range (min, max). The outlier is normalized using:

$$X' = \frac{x - min(x)}{max(x) - min(x)} \tag{5}$$

These processed data are used for training GRU network prediction model to identify anomalous data.

2. Multistep-Prediction

The proposed model identifies anomalies based on multi-step prediction. It compares actual data with predicted data and considers an anomaly if the difference between these two data is greater than a threshold value. The GRU is a type of Recurrent Neural Network (RNN) used to learn temporal data interference like time-series

UNCORRECTED PROOF

511082_1_En_9_Chapter ☑ TYPESET ☐ DISK ☐ LE ☑ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

data prediction and find a particular pattern in time series data. GRU is stronger than RNN, as it resolves the problem vanishing gradient, learns from inputs that are independent of each other, and can store longer memories. GRU networks are well suited to analyzing, classifying, and making forecasts based on time series data, as there can be unfamiliar period lags between significant events in time series [11]. So in this paper, the GRU network model is used to for multistep time series prediction. Figure 4 shows the GRU unit with its components.

The GRU has two gates: update and reset gate. These two gates regulate the information/data flow inside the GRU cell. An input gate allows only necessary input values in the GRU cell. It decides what information to remove and what new information to be added. The reset gate is used to decide how much past information to forget. Initially update gate z_t for time step t is calculated using following formula.

$$Z_t = \sigma(Wt_f.X_t + U^{(z)}h_{t-1}] \tag{6}$$

When *x_t* is plugged into the network unit, it is multiplied by its own weight *W(z)*. The same goes for *h_(t-1)* which holds the information for the previous *t-1* units and is multiplied by its own weight *U(z)*. Both results are added together and a sigmoid activation function is applied to squash the result between 0 and 1. next reset gate is used to decide how much past information to remove the network. The reset gate is can be calculated using

$$r_t = \sigma(Wt_f.X_t + U^{(r)}h_{t-1}] \tag{7}$$

the reset gate formula is similar to update gate. The weight and gate's usage are different in both gates. the current memory content is introduced to reset gate to store the relevant information from the past. the current memory content is calculated as follows:

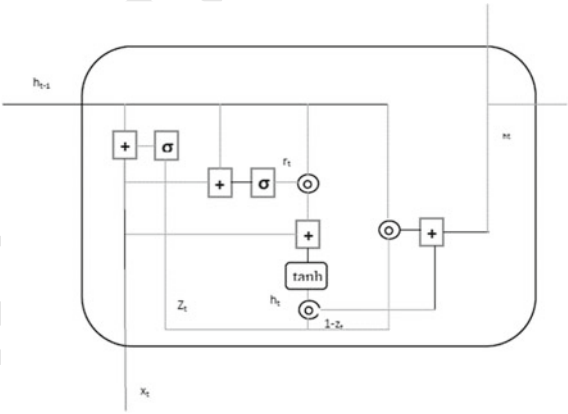

Fig. 4 GRU model

UNCORRECTED PROOF

511082_1_En_9_Chapter ✓ TYPESET ☐ DISK ☐ LE ✓ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

$$h_t = tan_h(W_x.O_t + r_i.Uh_{t-1}) \tag{8}$$

as the last step, the network needs to calculate h_t vector which holds the information for the current unit and passes it down to the network. it also determines what to collect from the memory content. The following formula identifies the important information in memory content.

$$h_t = Z_t.h_t + (1 - z_t).h_t^{'} \tag{9}$$

1. Apply element-wise multiplication to the update gate z_t and h_(t-1).
2. Apply element-wise multiplication to (1-z_t) and h'_t.
3. Sum the results from step 1 and 2.

The model can learn to set the vector z_t close to 1 and keep a majority of the previous information. Since z_t will be close to 1 at this time step, 1-z_t will be close to 0 which will ignore big portion of the current content (in this case the last part of the review which explains the book plot) which is irrelevant for our prediction.

In order to prevent the units from being strongly co-adapted, it is necessary to drop a few secret units along with their relationship during the training process. After dropout, each secret device will become more versatile and will be able to build valuable functionality on its own without relying on other units to help minimize overfitting. The dropout technique is used to reduce overfitting problem and improve the performance of neural networks applications. In the proposed model, dropout is used only in hidden layers. The main idea is to feed the normal data to GRU based Neural Network for training the GRU network to predicts the next 'n' time steps. The predicted values are compared to actual values and considered anomalous if the difference is larger than the specified threshold. Based on the density of anomalous points, it is categorized as warning, error, and issue.

The multi-step time-series prediction model is designed using GRU Model. Given existing time series data, $X = \{x_1, x_2 \ldots x_{t-1}\}$, where x_i denotes values at the time-stamp *i*, the task is to predict the future value x_t. The sliding window model is adopted to collect the recent values for multi-step prediction. For the prediction $\{x_{t-w}, x_{t+1-w} \ldots x_{t-1}\}$ is used, where 'w' is the window size. These sequences are divided into mini-batches and fed into GRU network for training with 'n' epochs.

$$y_{t+n} = O_n(y_t, y_{t+1}, y_{t+2} \ldots . y_{t+n-1}) \tag{10}$$

where "n" is the number of steps to forecast into the future, O_n is any GRU Learning model outcome. Figure 5 shows a window-based multistep prediction. The main advantage of GRU is it remembers existing values while predicting future values. The window size is 10 and sliding towards the right side 'n' step after predicting last value. The process will continue until specified number of multistep prediction is required.

UNCORRECTED PROOF

511082_1_En_9_Chapter ✓ TYPESET ☐ DISK ☐ LE ✓ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

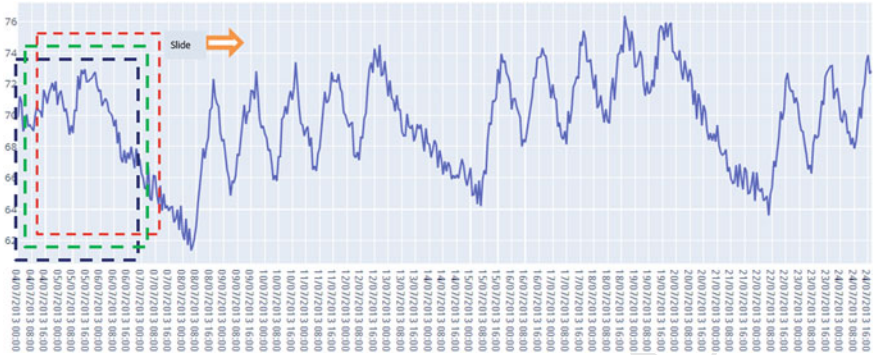

Fig. 5 Multistep prediction

2.1. **Hyperparameter optimization using Artificial Bee Colony.**

The ABC is a bio-inspired algorithm mimics bee colonies' intelligent behavior in foraging [13]. It has been used in various application to optimize the performance. The colony of bees in ABC is classified into three types. Employee bee searches the food around of food sources in their memory and shares knowledge to other bees, onlooker bee uses this knowledge and find the good quality of nectar. Scout bees find new food sources and converted themselves to employee bees. The proposed model uses ABC algorithm for hyperparameter tuning to improve the accuracy.

The hyperparameters are used to control the learning process in machine learning applications [15]. The performance of the model can be improved by optimizing hyperparameter values. The main objective of ABC optimization techniques is to choose the best combination values of hyperparameters to improve predictions' accuracy. Automatic Hyperparameter optimization has several advantages: first, it reduces human efforts in deploying machine learning models as a manual assigning hyperparameter to consume more computational resources, and it is a tedious process. The Hyperparameter configurations are dependent on the type of problem and its complexity. Second, the machine learning model improved performance by identifying the most suitable Hyperparameter values for the target application. Further, it improves overall performance with more reproducible results.

In the proposed anomaly detection model, a bio-inspired algorithm called an Artificial Bee Colony (ABC) algorithm is used for optimizing the GRU to increase the accuracy of prediction. The ABC algorithm is used to select appropriate hyperparameters of the GRU model. In ABC Algorithm, solutions to the optimization problems [15]. The Bees are always looking at storing good quality honey for a longer time. The quality of nectar is the main objective feature in processing and storing. The 'n' number of bees are employed for 'n' number of food sources. One onlooker bee is employed for each bee to identify the nectar's quality by taking information from the employed bee. The GRU model adopts ABC foraging behavior to select the best combination of hyperparameters to get the highest accuracy in multistep

UNCORRECTED PROOF

511082_1_En_9_Chapter ✓ TYPESET ☐ DISK ☐ LE ✓ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

prediction. The traditional ABC algorithm faces some challenges in exploration and exploitation.

Algorithm: Anomaly detection with hyperparameter optimization using Artificial Bee Colony

1: Initialize the Hyperparameter value (min, max) values, window size, time-steps, Employee Bees (EB), Onlooker Bees(OB), Maximum Cycle Number MCN), Solutions (S_N), k=0, and set the threshold value (0.5)

2. Create Initial population HP_x={SLW, N, DR, LR, RR, OP} and assign predefined standard value

$$x_{ij} = predifined_value$$

3. Read input values to sliding windows

4. Initialize the counter= 0

5. ***do***

Counter++

Assign sliding window values to the GRU network

Train the GRU network

Evaluate the fitness value of training using

$$fitness_i = \{\frac{1}{(1+f_i)} \; if \; f_i \geq 0 \; 1 + abs(f_i) if \; f_i < 0$$

$$p_i = \frac{fitness_i}{\sum_i^{SN} fitness_i} \qquad \text{for i = 1.. EB}$$

Check fitness values the previous solution

if (p_i<threshold) then

break

else

create a new solution by modifying an old solution (hyperparameters)

$$v_{ij} = x_{ij} + \emptyset_{ij}\,(x_{ij} - x_{kj})$$

While(counter >=MCN)

6. Memorize the best hyperparameter combination.

7. Get multi-step prediction values from trained GRU model

8. Calculate the error vector, mean and standard deviation of errors.

9. Product error vector

10. Estimate the threshold values based on error vector

11. do

if SD >= threshold then

k++

while (timesteps)

12. if k==1 then

Print "Warning/Point Anomaly"

Elsif k==2

Print "Error/Collective Anomaly"

Else

Print "Issue/Collective Anomaly"

Endif

To accelerate the convergence speed of ABC, a population size reduction strategy is introduced in standard ABC algorithm. It will help to increase the explorative search behavior at early stages of the search process. Based a survival strategy and survival rate (SR), it determines the which candidate solutions form the current population survive and pass to the next population. The SR is a constants value and its range is 0 to 1. The following equation represents the population search strategy:

511082_1_En_9_Chapter ☑ TYPESET ☐ DISK ☐ LE ☑ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

$$NP_{current} = \begin{cases} Max((NP_{current} - NP_{min}), NP_{threshold} & ifSR > r \\ NP_{current} & otherwise \end{cases} \quad (11)$$

Further, a modified search strategy is used to improve exploitation behavior of ABC. A new search operator uses information from the food sources of the present population by selecting random pair of information and also best food source. The following equation represents the new search operator

$$v_{ij} = x_{ij} + sf * (x_{best,j} - x_{ij} + x_{r1,j} - x_{r2,j}) \quad (12)$$

The hyperparameter in proposed work are: (i) sliding-window-size (oh1), GRU-units (oh2), dropout-rate (oh3), regularizer (oh4), regularizer-rate (oh5), optimizers (oh6), epochs (oh7) and learning rate (oh8). Each hyperparameter has different types and ranges of values. The solutions are obtained in the form of the best amalgamation of hyperparameters that gives the highest accuracy for multi-step prediction problem. These hyperparameter combinations are used to train the GRU network for multistep predictions. Further, the prediction model are evaluated using RMSE and it is considered as the fitness value in ABC algorithm. The ABC algorithm initially assigns the values to the hyperparameter. The GRU model trained with these hyperparameters and calculate the fitness value (RMSE). At the next step, new random hyperparameters values are assigned and check fitness value. The algorithm works iteratively until maximum fitness value is found or maximum cycle numbers (iterations) completed. The new fitness value is recorded if it is better than previously computed fitness value.

3. Anomaly Detection Model

The multi-step prediction is used to identify the anomalous data in the temporal data. The observed data are compared with predicted values. If the error rate between the observed value and the predicted value is greater than a predefined threshold value, then it is considered an anomaly. The threshold value varies depending upon the type of application. The following equation is used to calculate the error:

$$E_t = |y_t - x_t| \quad (13)$$

where x_t is the timestamp data, $X = (x_1, \times 2, \ldots x_m)$, y_t is the value of timestamp t in the time series data $Y = (y_1, y_2, \ldots y_m)$, and E_t is error value. These prediction error are put together in a vector form. The threshold value *Tp* is defined to find the anomalies in temporal data using following equation. The Error rate is higher than threshold T_p then it is considered as anomalous value(s):

$$A = E_t > T_p \quad (14)$$

Based on number of anomalies identified, it is classified 'warning',' error' and 'issue'. The proposed model classifies the anomalies as warning, error, or issue depending upon the number of anomalous points identified in the temporal data.

UNCORRECTED PROOF

Author Proof

The single anomaly point is considered as 'warning' and two anomaly points are considered as 'error'. More than two anomalies are considered as 'issue'. The overall anomaly detection process is shown in Fig. 7. The anomaly score is used to identify significance of problem and it is defined as:

$$S_x = \sqrt{(\tilde{x} - x)^T \sum_{.}^{-1} (\tilde{x} - x)}, \tag{15}$$

where, where Σ is the 'covariance matrix' of the training input, x is the input values and ~x is predicted value. The spike and drift are the common anomalies in time series dataset. To identify spike and drift the following equation are used. The prediction error is used to identify the spike and drift anomalies. The prediction error is more than threshold and it has highest variance then it is considered as *spike*. The prediction error range have the same pattern with high prediction is considered as *drift*. The spike and drift are calculated using following equations

$$Spike_t = E_t > T_p AND E_t > Var_{max} \tag{16}$$

Here, $\mathrm{Var}_{\max}$ is maximum variation, E_t is error rate at time t and T_p is threshold value.

$$Drift_t = \left[Avg(W_{i-1}) - Avg(W_1)\right] > n \tag{17}$$

Here, the window values are used to calculate the drift values. The difference between previous average values of window and current average values of windows is greater than *'n'* is considered as drift. Depending upon the types of data and its application, the value of n is set.

4 Performance Evaluation

This section discusses experiment setup, performance evaluation of the proposed anomaly detection model. The proposed model is compared with start of art anomaly detection models.

Experiment Setup

The experiment was evaluated on Google Colab with 2 GB GPU, 16 GB RAM, and 160 GB Hard disk space, The proposed model is implemented using Python programming language with TensorFlow Keras library. Numenta Anomaly dataset is used to test the efficiency of the proposed anomaly detection model.

UNCORRECTED PROOF

511082_1_En_9_Chapter ☑ TYPESET ☐ DISK ☐ LE ☑ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

Dataset

The Numenta Anomaly Benchmark is a dataset (NAB Dataset. n.d) developed to evaluate anomaly detection algorithms in various real-time and streaming applications. The datasets in NAB (Ahmad, Lavin, Purdy & Agha 2017), namely CPU utilization as misconfiguration, ambient temperature system failure, and machine temperature system failure are used for experiment purposes. This dataset contains a timestamp and its respected values like temperature and CPU consumption etc. Data normalization is a technique used during the data preparation phase for machine learning models. The normalization process the scale the values in the range of [0,1] without obfuscating its values. Figure 6 shows several datasets from NAB benchmark that exhibits short and long term temporal dataset.

Hyperparameters

Eight hyperparameters are used to tune the GRU based multi-step prediction model. These hyperparameters are i) sliding-window-size (oh1), GRU-units (oh2), dropout-rate (oh3), regularizer (oh4), regularizer-rate (oh5), optimizer (oh6), epochs (oh7), activation function (oh8) and learning-rate (oh9). The most important hyperparameter are learning rate, epochs, and dropout rate because there is a direct impact on model performance.

The performance metrics

To evaluate the proposed model, the performance metrics: precision, recall, and F1-score are used. These metrics represent the accuracy and efficiency of the anomaly detection process. To calculate these metrics it is essential to understand statistical

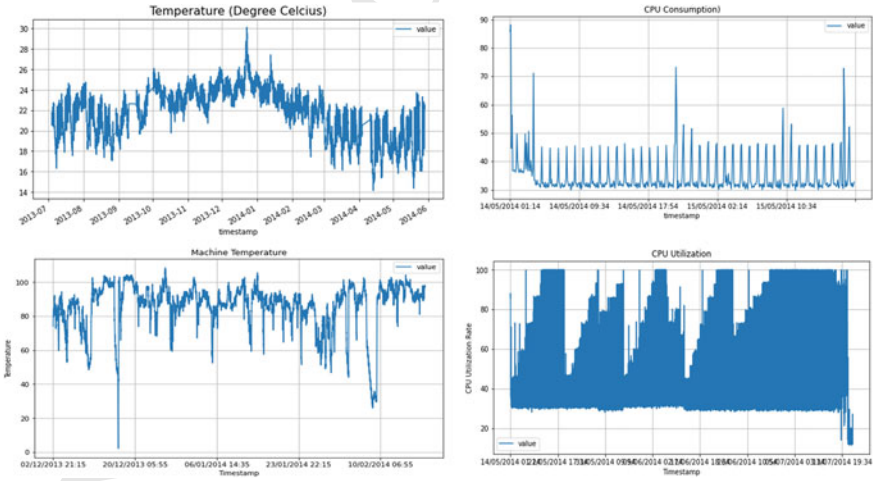

Fig. 6 Sample NAB dataset (i) temperature and (ii) CPU consumption values (iii) Machine temperature sensor data (iv) CPU utilization as misconfiguration

UNCORRECTED PROOF

511082_1_En_9_Chapter ✓ TYPESET ☐ DISK ☐ LE ✓ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

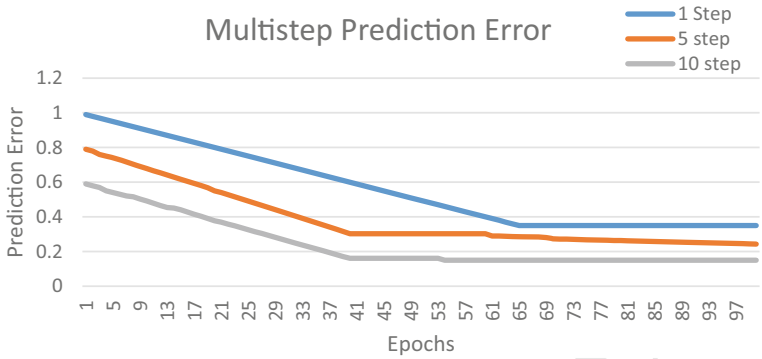

Fig. 7 Comparison of prediction error for various multi-step predictions

measures of the performance of binary classification: sensitivity and specificity. The various performance metric are:

(i) Precision, P, is the percentage of correctly identified values. In other words, it is a measure of correctly identified anomalies.

$$precision = \frac{TruePositive}{TruePositive + FalsePositive} \tag{18}$$

Recall, R, which is the number of positive class prediction out of all positive examples,

$$Recall = \frac{TruePositive}{TruePositive + FalseNegative} \tag{19}$$

and (iii) F1-score, F1, which is the harmonic mean of Precision and Recall, and it provides a more accurate picture of cases that were incorrectly classified than the Accuracy Metric.

$$F1Score = \frac{2 * (Precision * Recall)}{Precision + Recall} \tag{20}$$

The F1-score represents the balance between precision and recall and it summarizes both. The detection rule with the highest F1 as the superior anomaly detection technique. The accuracy is another important key performance parameters to evaluate the anomaly detection model. The accuracy is defined as follows

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{21}$$

UNCORRECTED PROOF

Author Proof

$$ErrorRate = \frac{FN + FP}{TP + FP + TN + FN} \tag{22}$$

The various loss functions are defined for machine learning models such as 'Root Mean Squared Error' (RMSE)', 'Mean Squared Logarithmic Error Loss (MSLEL)', and 'Mean Absolute Error Loss (MAE)', etc. The RMSE is the right choice to evaluate time series prediction model, and it is defined as

$$Root\ Mean\ Squared\ Error = \sqrt{\frac{1}{n}\sum\nolimits_{i=1}^{n}(y^i - y_p^i)^2} \tag{23}$$

The larger RMSE indicates less accuracy and least RMSE indicates highest accuracy. The various existing works are compared to the proposed model to evaluate the performance of the model. Traditional models and machine learning model for multi-time step prediction includes 'Auto-Regressive Integrated Moving Average (ARIMA)'[12], 'Hidden Markov Model' [11], 'Support Vector Machine model' [8], GRU autoencoder model [13].

5 Results

In this section, the results of the proposed model are discussed. This section consists of two subsections (i) results of the proposed model and (ii) comparison of the proposed model with few existing models. Prediction accuracy, anomaly detection accuracy, and RMSE are discussed. The multi-step prediction is used to find the anomalies in the proposed model. The difference between predicted values and the actual value is more than a threshold is considered as an anomaly. The GRU network models are trained with different time-step from (1 to 10) to predict the future values of time series data. Figure 8 shows the prediction error for multi-step predictions. The accuracy decreases as the number of time-steps are increases. To improve accuracy, it is essential to take multiple time-steps predictions. The accuracy of anomaly detection is dependent on multi-step ahead prediction accuracy. So the proposed model considers timesteps: 5. Figure 7 shows the multistep prediction with different epochs.

The sliding window approach is used for multi-step prediction, where various window sizes (window size: 1, 5, 10, 20) are used to predict multi-step values. The following graphs show the comparison of predicted values and test values for different window sizes. It has been observed that the prediction accuracy increases as the size of the window increases. For the multi-step prediction, n input is fed to the window to predict n + 1. The accuracy increases as more inputs fed to the window. So windows size is directly proportional to accuracy. Figure 8 shows the prediction accuracy for different window sizes.

To evaluate the anomaly detection model, the loss function RMSE is calculated for test dataset and validation. Figure 9 shows the difference between train and test

UNCORRECTED PROOF

511082_1_En_9_Chapter ✓ TYPESET ☐ DISK ☐ LE ✓ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

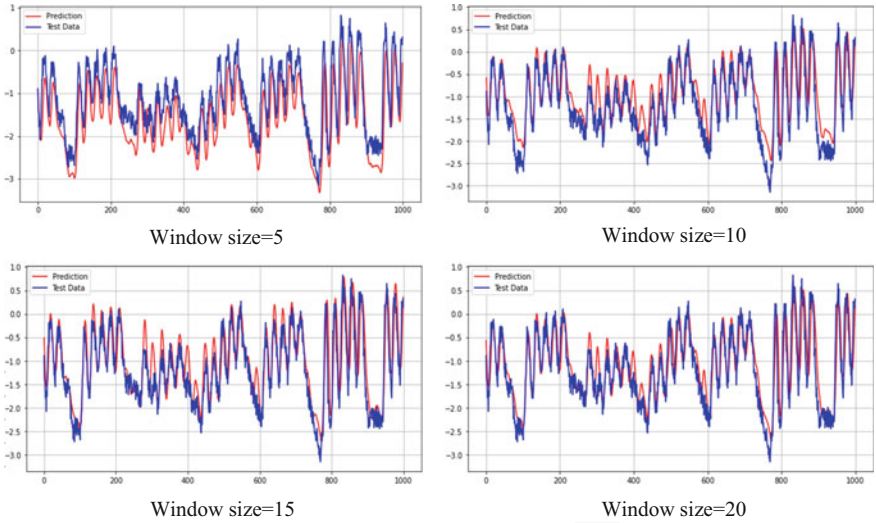

Fig. 8 Comparison between predicted data and test data for different window size

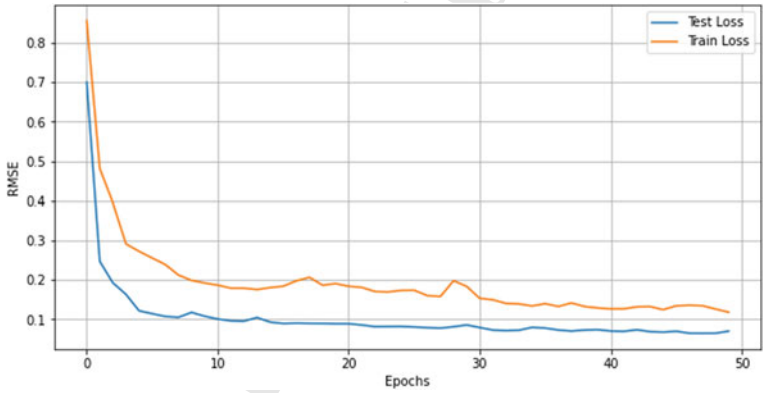

Fig. 9 RMSE for train and test dataset

loss for different Epochs. The difference between test and train loss is very less, and the loss is reduced as the number of epochs in increases. From the graph, it has been observed that the accuracy of the proposed anomaly detection model is at the higher side.

Figure 10 shows the comparison of RMSE for various anomaly detection models. The less RMSE represents higher accuracy. The proposed model has less RMSE compared to existing anomaly detection models.

The accuracy of the proposed model has been increased compared to the existing models, but more memory and CPU cycles required for hyperparameter tuning. So

511082_1_En_9_Chapter ✓ TYPESET DISK LE ✓ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

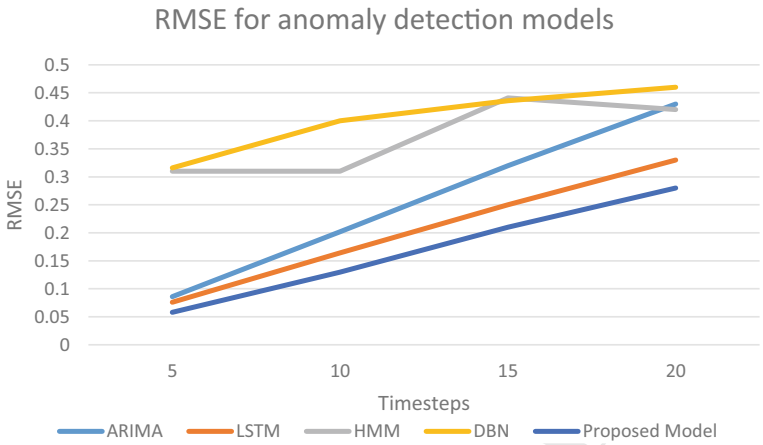

Fig. 10 RMSE comparison of existing model versus the proposed model

proposed model can be adopted in critical applications where accuracy is the highest priority. Figure 11 shows the comparison of various performance parameters such as CPU consumption, memory consumption, anomaly detection time, and accuracy.

Figure 12 shows the recall, precision, and F1-score comparison of existing anomaly detection models with the proposed model. The existing models are results are taken from RNN, GRU [12], and auto-encoder [16]. The proposed model achieves higher precision, recall, and F1-score when compared with the other three methods.

Table 1 shows that the hyperparameter combination with the threshold value 0.65 for which the accuracy obtained is 98.8. It has been observed from the experiment

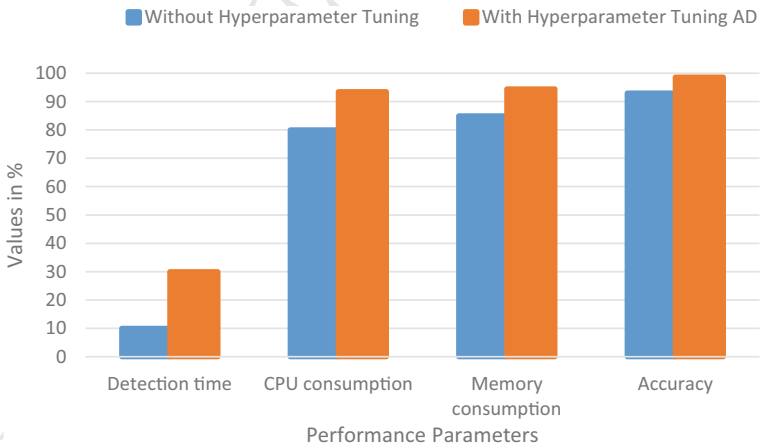

Fig. 11 Comparison of normal AD and hyperparameter tuner AD

511082_1_En_9_Chapter ✓ TYPESET DISK LE ✓ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

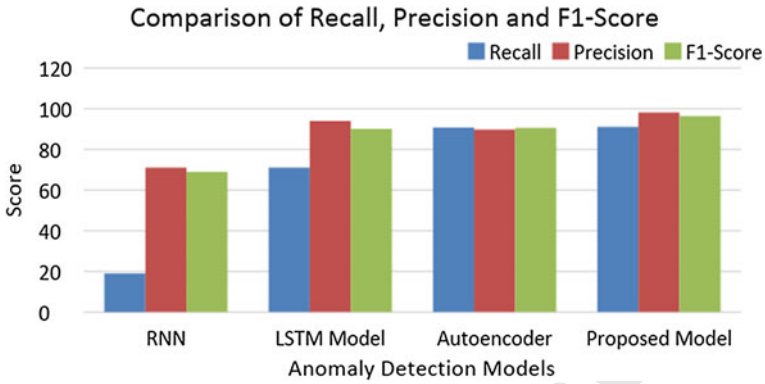

Fig. 12 Precision, recall, and F-score comparison of the existing and proposed system

Table 1 Hyperparameter values for highest accuracy

Hyperparameters	Values
Epochs	100
No. of Units	30
Dropout Rate	0.3
Regularizer	L2
Regularizer rate	0.01
Optimizer	RMSprop
Learning Rate	0.09
Activation function	ReLu

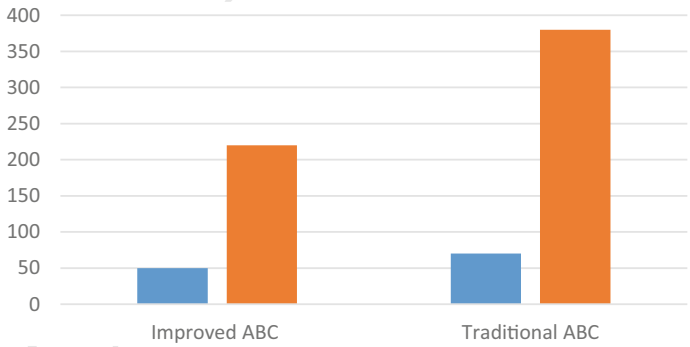

Fig. 13 Comparison of traditional and improved ABC algorithm

results; hyperparameter optimization gained the highest accuracy compared to other models.

UNCORRECTED PROOF

511082_1_En_9_Chapter ☑ TYPESET ☐ DISK ☐ LE ☑ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

Figure 13 shows the different between traditional and improved algorithm. During experiment evaluation, improved ABC takes less time in search and computation time.

6 Conclusion

Anomaly detection is one of the key aspects to improve the Quality of Service through the failure prediction. In this paper, a multi-agent based anomaly detection model is introduced to detect anomalies in the Industrial IoT. Multi agent system is used to perform various operations in the anomaly detection process. A deep learning model called the GRU model is used for multi-step prediction. The proposed model considers certain timestamp value as an anomaly if the difference between actual value and predicted value is greater than threshold value. To improve the accuracy of prediction, the GRU model adopted a bio-inspired algorithm called Artificial Bee Colony (ABC) for tuning hyperparameters. The proposed model is evaluated in the google Colab environment with the NAB temperature and CPU utilization data set. The experiment result shows that the proposed model improves accuracy compared to existing anomaly detection models. In future studies, the detection time can be improved by using new optimization techniques.

References

1. M.V. Ngo, T. Luo, H. Chaouchi, T.Q.S. Quek, Contextual-bandit anomaly detection for IoT data in distributed hierarchical edge computing, in *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)* (2020), pp. 1227–1230. doi: https://doi.org/10.1109/ICDCS47774.2020.00191
2. C.M. Bulla, M.N. Birje, A multi-agent-based data collection and aggregation model for fog-enabled cloud monitoring. Int. J. Cloud Applicat. Comput. **11**, 73–92 (2021)
3. M.V. Ngo, H. Chaouchi, T. Luo, T.Q.S. Quek, Adaptive anomaly detection for IoT data in hierarchical Edge Computing. arXiv:2001.03314 [cs, stat]. (2020)
4. Utomo, D., Hsiung, P.-A.: Anomaly Detection at the IoT Edge using Deep Learning. In: 2019 IEEE ICCE-TW. pp. 1–2. IEEE, YILAN, Taiwan (2019)
5. M.N. Birje, C.M. Bulla, Cloud monitoring system: basics, phases and challenges. Int. Jou. Rcnt. Tech. & Engg. **8**, 4732–4746 (2019)
6. S. Thudumu, P. Branch, J. Jin et al., A comprehensive survey of anomaly detection techniques for high dimensional big data. J Big Data **7**, 42 (2020). https://doi.org/10.1186/s40537-020-00320-x
7. J. Wu, L. Yao, B. Liu, Z. Ding, L. Zhang, Combining OC-SVMs with GRU for detecting anomalies in telemetry data with irregular intervals. IEEE Access. **8**, 106648–106659 (2020)
8. M. Birje, C. Bulla, Cloud monitoring system: a review. Int. J. Sci. Techn. Eng. Mgt. **1**(1), 49–55, (2019)
9. C. Mitchell, R. Agrawal, J. Parker, The effectiveness of edge centrality measures for anomaly detection, in *2019 IEEE International Conference on Big Data (Big Data)* (2019), pp. 5022–5027. doi: https://doi.org/10.1109/BigData47090.2019.9006468

UNCORRECTED PROOF

Author Proof

10. S. Maya, K. Ueno, T. Nishikawa, dGRU: a new approach for anomaly detection using deep learning with delayed prediction. Int. J. Dat. Sci. Ana. **8**, 137–164 (2019)
11. Y. Tan, C. Hu, K. Zhang, K. Zheng, E.A. Davis, J.S. Park, GRU-based anomaly detection for non-linear dynamical system. IEEE Access. **8**, 103301–103308 (2020). https://doi.org/10.1109/ACCESS.2020.2999065
12. Y. Gao, An improved hybrid group intelligent algorithm based on artificial bee colony and particle swarm optimization, in *2018 International Conference on Virtual Reality and Intelligent Systems (ICVRIS)* (2018), pp. 160–163. doi: https://doi.org/10.1109/ICVRIS.2018.00046
13. M. Said Elsayed, N.-A. Le-Khac, S. Dev, A.D. Jurcut, Network anomaly detection using GRU based autoencoder, in *Proceedings of the 16th ACM Symposium on QoS and Security for Wireless and Mobile Networks*, pp. 37–45 (2020)
14. B. Laughlin et al., A service architecture using machine learning to contextualize anomaly detection. J. Database Manag. **31**(1), 64–84 (2020). https://doi.org/10.4018/JDM.2020010104
15. A. Alghuried, *A Model for Anomalies Detection in Internet of Things (IoT) Using Inverse Weight Clustering and Decision Tree* (Technological University, Dublin, 2017)

UNCORRECTED PROOF

511082_1_En_9_Chapter ☑ TYPESET ☐ DISK ☐ LE ☑ CP Disp.:27/8/2021 Pages: 22 Layout: T1-Standard

Author Proof

Author Queries

Chapter 9

Query Refs.	Details Required	Author's response
AQ1	Please note that the figures and table are renumbered to ensure sequential order of citations. Please check and confirm the change.	

UNCORRECTED PROOF

MARKED PROOF

Please correct and return this set

Please use the proof correction marks shown below for all alterations and corrections. If you wish to return your proof by fax you should ensure that all amendments are written clearly in dark ink and are made well within the page margins.

Instruction to printer	*Textual mark*	*Marginal mark*
Leave unchanged	under matter to remain	
Insert in text the matter indicated in the margin		New matter followed by or
Delete	through single character, rule or underline or through all characters to be deleted	or
Substitute character or substitute part of one or more word(s)	through letter or through characters	new character or new characters
Change to italics	under matter to be changed	
Change to capitals	under matter to be changed	
Change to small capitals	under matter to be changed	
Change to bold type	under matter to be changed	
Change to bold italic	under matter to be changed	
Change to lower case	Encircle matter to be changed	
Change italic to upright type	(As above)	
Change bold to non-bold type	(As above)	
Insert 'superior' character	through character or where required	or under character e.g. or
Insert 'inferior' character	(As above)	over character e.g.
Insert full stop	(As above)	
Insert comma	(As above)	
Insert single quotation marks	(As above)	or and/or or
Insert double quotation marks	(As above)	or and/or or
Insert hyphen	(As above)	
Start new paragraph		
No new paragraph		
Transpose		
Close up	linking characters	
Insert or substitute space between characters or words	through character or where required	
Reduce space between characters or words	between characters or words affected	